INTEGRAÇÃO E ENTREGA CONTÍNUA

Gerenciamento de Configuração

Professora: Lucineide Pimenta

Tópicos da aula

- As ferramentas de CD/CI Jenkins e GitLab CI/CD.
- Escolha a melhor ferramenta, GitLab ou Jenkins.

INTEGRAÇÃO E ENTREGA CONTÍNUA

Escolha a melhor ferramenta, GitLab ou Jenkins

Escolha a melhor ferramenta, GitLab ou Jenkins

- O Jenkins é uma das ferramentas mais antigas e populares para CI/CD.
- É um servidor de automação open-source que facilita a construção, teste e entrega contínua de software.
- O Jenkins é amplamente utilizado devido à sua flexibilidade e ao vasto ecossistema de plugins.

Escolha a melhor ferramenta, GitLab ou Jenkins

- **Características Principais:**

1. Open-source: O Jenkins é gratuito e mantido por uma vasta comunidade de utilizadores e desenvolvedores.

2. Extensibilidade via plugins: O Jenkins tem mais de 1.800 plugins disponíveis, o que permite personalizar pipelines de CI/CD para se adequar a praticamente qualquer linguagem, ferramenta ou serviço de cloud.

Escolha a melhor ferramenta, GitLab ou Jenkins

3. Pipeline-as-Code: Com o Jenkins Pipeline, pode-se definir todo o processo de integração e entrega contínua num arquivo de configuração (geralmente escrito em Groovy), tornando os pipelines fáceis de versionar e replicar.

4. Suporte para várias plataformas: Jenkins pode ser executado em qualquer sistema operativo que suporte Java (Windows, Linux, macOS), o que o torna extremamente flexível.

5. Automatização completa: Jenkins permite automatizar não só a construção e os testes, mas também a entrega e a implementação de software em ambientes de produção.

Escolha a melhor ferramenta, GitLab ou Jenkins

- **Vantagens:**
  - - Altamente personalizável: Pode-se adaptar o Jenkins a quase qualquer tipo de fluxo de trabalho.
  - - Ampla comunidade de suporte: Devido à sua popularidade, há uma grande quantidade de documentação, tutoriais e suporte disponíveis.
  - - Suporte a múltiplos idiomas e tecnologias: Jenkins pode ser usado em projetos em praticamente qualquer linguagem de programação ou arquitetura de software.

Escolha a melhor ferramenta, GitLab ou Jenkins

- **Desvantagens:**

  - Configuração complexa: Para utilizadores iniciantes, a configuração do Jenkins pode ser um pouco desafiante devido à sua flexibilidade.

  - Necessidade de manutenção: Sendo uma ferramenta tão personalizável e extensível, pode necessitar de mais atenção em termos de atualizações e manutenção de plugins.

Escolha a melhor ferramenta, GitLab ou Jenkins

- **Cenários de Uso:**

  Projetos grandes ou heterogéneos: Ideal para equipes que trabalham em grandes sistemas que envolvem múltiplas tecnologias.

  - Ambientes altamente personalizados: Para equipes que necessitam de uma configuração de pipeline muito específica.

Escolha a melhor ferramenta, GitLab ou Jenkins

- **Exemplo de workflow no Jenkins:**

1. Um programador faz um commit para o repositório Git.

2. Jenkins automaticamente detecta a alteração, inicia o processo de build e executa testes automatizados.

3. Se o build e os testes forem bem-sucedidos, Jenkins pode automaticamente disponibilizar o software para um ambiente de staging ou produção.

Escolha a melhor ferramenta, GitLab ou Jenkins

- **GitLab CI/CD**

  - O GitLab CI/CD é uma solução de integração e entrega contínua que está integrada diretamente no GitLab, uma plataforma de desenvolvimento colaborativo baseada em Git.

  - Ao contrário do Jenkins, o GitLab CI/CD faz parte de um ecossistema unificado que inclui repositório de código, rastreamento de problemas, revisões de código e CI/CD, tudo numa única interface.

Escolha a melhor ferramenta, GitLab ou Jenkins

- **Características Principais:**

  1. Integração com GitLab: Uma das principais vantagens do GitLab CI/CD é que ele está profundamente integrado com o repositório GitLab. Isso significa que não precisas de configurar servidores separados para CI/CD.

  2. Pipelines-as-Code: No GitLab CI/CD, os pipelines são definidos num arquivo *'.gitlab-ci.yml'* na raiz do repositório. Este arquivo especifica todas as fases (build, teste, deploy) e define como devem ser executadas.

  3. Suporte nativo a containers e Kubernetes: O GitLab tem suporte integrado para containers Docker e pode-se integrar facilmente com plataformas como Kubernetes, o que facilita a entrega e escalabilidade de aplicações baseadas em microserviços.

Escolha a melhor ferramenta, GitLab ou Jenkins

4. Runner integrado: GitLab CI/CD usa GitLab Runners, que são executores de tarefas que realizam os trabalhos definidos nos pipelines. Pode, com isso, configurar runners específicos para diferentes tipos de trabalho (builds, testes, deploys, etc.).

5. Interface unificada: Como a CI/CD faz parte da mesma interface GitLab, pode-se facilmente navegar entre o código, as pipelines de CI/CD, os testes e as implementações, tudo num só lugar.

Escolha a melhor ferramenta, GitLab ou Jenkins

- **Vantagens:**

  Fácil de configurar: Comparado ao Jenkins, o GitLab CI/CD é mais fácil de configurar para utilizadores que já usam GitLab como repositório.

  - Plataforma unificada: Todas as ferramentas para desenvolvimento, desde o código até ao deployment, estão integradas na mesma plataforma.

  - Pipelines-as-Code simplificados: O arquivo '*.gitlab-ci.yml*' é bastante fácil de entender e configurar, mesmo para programadores com menos experiência em CI/CD.

  - Escalabilidade: Excelente para equipes que querem escalar facilmente utilizando containers e Kubernetes.

Escolha a melhor ferramenta, GitLab ou Jenkins

- **Desvantagens:**

  - Menos flexível que o Jenkins: O GitLab CI/CD é poderoso, mas pode ser menos flexível em comparação com o Jenkins em certos cenários altamente personalizados.

  - Dependência do ecossistema GitLab: Embora o GitLab CI/CD funcione bem com GitHub e outros repositórios, é mais natural para equipas que já usam GitLab.

Escolha a melhor ferramenta, GitLab ou Jenkins

- **Cenários de Uso:**

  - Equipes que usam GitLab: Se já usas o GitLab como repositório de código, faz sentido usar o CI/CD integrado.

  - Equipes que preferem simplicidade: GitLab CI/CD é mais simples de configurar e manter do que o Jenkins para projetos mais lineares.

Escolha a melhor ferramenta, GitLab ou Jenkins

- **Exemplo de workflow no GitLab CI/CD:**

  1. Um programador faz um commit no GitLab.

  2. O GitLab CI/CD inicia automaticamente a pipeline definida no arquivo '.gitlab-ci.yml'.

  3. A pipeline pode incluir várias fases, como compilar o código, executar testes, e implantar numa aplicação.

  4. Dependendo do sucesso ou falha da pipeline, o GitLab fornece relatórios e feedback diretamente na interface de utilizador.

Escolha a melhor ferramenta, GitLab ou Jenkins

Jenkins vs. GitLab CI/CD

Característica	Jenkins	GitLab CI/CD
Tipo	Open-source, servidor autónomo de CI/CD	Parte integrada do GitLab (VCS + CI/CD)
Complexidade	Alta, flexível, mas pode ser complexo configurar	Mais simples de configurar, especialmente para utilizadores GitLab
Pipeline-as-Code	Jenkins Pipeline (Groovy)	`.gitlab-ci.yml` (YAML)
Extensibilidade	Mais de 1.800 plugins disponíveis	Menos extensível, mas tem funcionalidades integradas
Suporte a Containers	Via plugins como Docker e Kubernetes	Suporte nativo a Docker e Kubernetes
Interface	Necessário interface separada para CI/CD	Tudo num único interface (GitLab)

Escolha a melhor ferramenta, GitLab ou Jenkins

- ### **Jenkins vs. GitLab CI/CD**

- Jenkins é ideal para equipes que precisam de uma solução altamente personalizável e que tenham necessidades mais complexas de integração e entrega. Também é uma excelente opção se já usa outras ferramentas de controle de versão que não sejam GitLab.

- GitLab CI/CD, por outro lado, é ótimo para equipes que querem uma solução mais simplificada e integrada, especialmente se já estão a usar o GitLab como repositório de código.

Escolha a melhor ferramenta, GitLab ou Jenkins

- **Conclusão**

- Escolha GitLab CI/CD se:

  - Já utiliza o GitLab como repositório de código.
  - Prefere uma solução fácil de configurar e manter.
  - Não precisa de uma personalização extrema nos pipelines.
  - e, se procura minimizar o esforço de manutenção e administração.

Escolha a melhor ferramenta, GitLab ou Jenkins

- **Conclusão**

  Escolha Jenkins se:

  - Precisa de personalizar ao máximo o pipeline e o ambiente CI/CD.

  - Usa diferentes ferramentas de controle de versão (como GitHub ou Bitbucket) e quer flexibilidade na integração.

  - Está trabalhando com grandes projetos ou múltiplas equipes, cada uma com requisitos específicos.

  - Não se importa de investir mais tempo na configuração e manutenção.

Bibliografia Básica

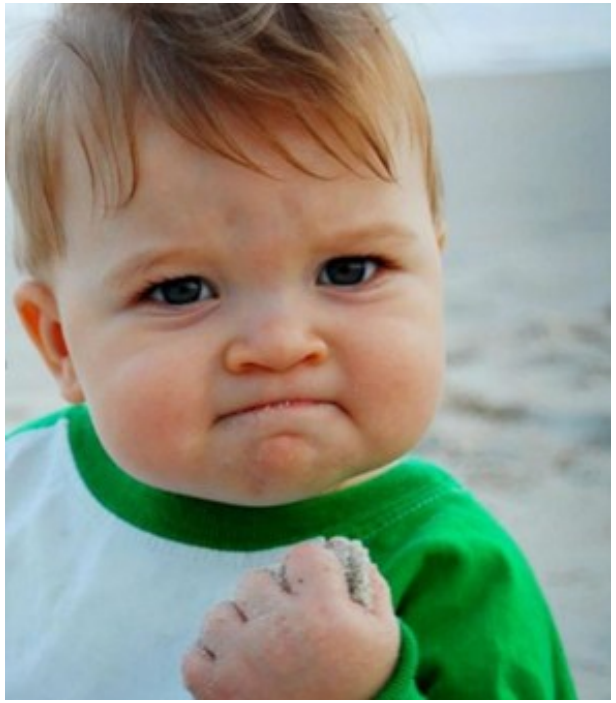
- HUMBLE J; PRIKLANDNICKI R. **Entrega Contínua:** Como Entregar Software de Forma Rápida e Confiável. São Paulo: Bookman, 2013.
- MUNIZ, A.; et al. **Jornada DevOps:** Unindo Cultura Ágil, Lean e Tecnologia Para Entrega de Software Com Qualidade. São Paulo: Brasport, 2019.
- SATO D. **DevOps na prática:** entrega de software confiável e automatizada. São Paulo: Casa do Código, 2014.
- SILVA, R. **Entrega contínua em Android:** Como automatizar a distribuição de apps. São Paulo: Casa do Código, 2016.

Bibliografia Complementar

- ARUNDEL, J. DOMINGUS, J. **DevOps nativo de nuvem com Kubernetes**. São Paulo: Novatec, 2019.
- MORAES, G. **Caixa de Ferramentas DevOps:** Um guia para construção, administração e arquitetura de sistemas modernos. São Paulo: Casa do Código, 2015. PIRES, A.; MILITÃO, J. **Integração Contínua com Jenkins**. São Paulo: Casa do Código, 2019.
- VITALINO, J. F. N.; CASTRO, M. A. N. **Descomplicando o Docker.** 2 ed. São Paulo: Brasport, 2018.
- SILVERMAN, R. E. **Git:** guia prático**.** São Paulo: Novatec, 2019.

Dúvidas?

Considerações Finais



Professora:

Lucineide Pimenta

Bom descanso à todos!